实验二：搜索算法实验报告

GitHub 仓库：https://github.com/shixuhui-ops/AI-Search-Experiment-2

一、实验目标

本实验实现并测试 A* 搜索、加权地形 A* 搜索、Minimax、Alpha-Beta 剪枝，以及结合 A* 和博弈搜索的宝藏分配问题。

实验重点是理解搜索状态、代价函数、启发函数、优先队列、极大极小递归和 Alpha-Beta 剪枝条件，并通过实验数据分析不同策略的性能差异。

二、算法设计

1. 迷宫 A*

状态为网格中的坐标 `(row, col)`。

- `g(n)`：从起点到当前节点的实际移动代价。
- `h(n)`：当前节点到终点的曼哈顿距离。
- `f(n) = g(n) + h(n)`。
- 每次从优先队列中取出 `f(n)` 最小的节点。
- 障碍物 `X` 不允许进入。
- 每次移动的代价为 1。

对于四方向移动，曼哈顿距离不会高估实际剩余步数，因此是可采纳启发函数，可以保证 A* 找到最短路径。

2. 加权地形 A*

加权地图中，进入相邻格子的代价等于目标格子的地形值，进入终点 `E` 的代价为 0。

算法仍使用优先队列进行搜索，并将累计地形代价作为 `g(n)`。无解时返回 `-1`。

由于进入终点 `E` 的代价为 0，直接使用完整曼哈顿距离可能高估实际剩余代价。因此实现首先计算地图中除终点外所有可通行格子的最小进入代价 `c_min`，再使用：

```
h(n) = max(曼哈顿距离(n, E) - 1, 0) × c_min
```

最后一步进入 `E` 的代价为 0，因此只计算此前需要付费的步数。由于其他可通行格子的进入代价均不低于 `c_min`，该启发函数不会高估真实剩余代价，具有可采纳性。在最低地形代价大于 1 的地图上，它比固定使用单位代价的曼哈顿距离更强。

3. Minimax

根节点为 MAX 玩家，下一层为 MIN 玩家，双方交替进行。

- MAX 节点选择子节点的最大值。
- MIN 节点选择子节点的最小值。
- 叶子节点直接返回对应分数。

4. Alpha-Beta 剪枝

Alpha-Beta 在 Minimax 基础上维护两个边界：

- `alpha`：MAX 当前已经找到的最好值。
- `beta`：MIN 当前已经找到的最好值。

当 `alpha >= beta` 时，剩余分支不可能改变祖先节点的最终决策，因此可以直接剪枝。

综合题的 `treasure_alpha_beta` 支持 `order_moves=True`。启用后，算法按照当前玩家获取宝藏的即时收益从高到低访问子节点，使较好的候选值更早更新 `alpha` 或 `beta`，从而增加后续剪枝机会。

性能结果：

深度	原始顺序	启发式排序	减少比例
H=2	13	11	15.4%
H=4	49	39	20.4%

节点排序不会改变最优结果：H=2 仍选择 T2、收益差为 -1；H=4 仍选择 T3、收益差为 3。

三、题目一：迷宫寻路结果

测试地图中，起点为 `(0, 0)`，终点为 `(3, 4)`。

最短路径为：

```
[(0, 0), (0, 1), (0, 2), (1, 2),
 (1, 3), (1, 4), (2, 4), (3, 4)]
```

路径包含 8 个坐标，实际移动 7 步。

沿该路径，`g(n)` 从 0 逐步增加到 7，曼哈顿距离 `h(n)` 从 7 逐步减少到 0，因此路径上各节点的 `f(n)=g(n)+h(n)` 均为 7。优先队列持续选择当前 `f` 值最小的节点，并通过 `parent` 记录前驱；到达终点后沿前驱指针逆向回溯得到完整路径。由于曼哈顿距离可采纳，首次取出终点时的 `g=7` 即为最短距离。

使用 `h=0` 时扩展 16 个节点，使用曼哈顿距离时扩展 14 个节点。两种方法得到相同的最优路径。

四、题目二：加权地形结果

加权地图中，进入不同地形的代价不同，进入终点 `E` 的代价为 0。

算法计算得到从起点到终点的最小总代价为 7。

对应的一条最小代价路径为：

```
(0,0) → (1,0) → (2,0) → (3,0) → (3,1)
→ (3,2) → (4,2) → (4,3) → (4,4)
```

各步进入代价依次为：

```
1 + 1 + 1 + 1 + 1 + 1 + 1 + 0 = 7
```

最后进入终点 `E` 的代价为 0。该路径避开了权值为 8 的高代价地形。

该结果说明加权搜索不能只比较路径长度，还需要比较经过地形的累计代价。某些路径虽然经过的格子更多，但由于避开了高代价地形，总代价反而更低。

五、题目三：Minimax 与 Alpha-Beta 结果

在题目给出的博弈树上，Minimax 与 Alpha-Beta 得到的最优值均为 5。Minimax 访问 15 个节点，Alpha-Beta 访问 14 个节点，剪去 1 个不可能影响最终决策的节点。

两种算法结果一致，说明 Alpha-Beta 不会改变 Minimax 的正确结果。其效率提升来自 `alpha >= beta` 时提前停止当前分支的后续搜索。

搜索树的数值回传过程如下：

```
MAX(root)
├── MIN
│   ├── MAX(3, 5) = 5
│   └── MAX(6, 9) = 9
│       MIN = min(5, 9) = 5
└── MIN
    ├── MAX(6, 9) = 9
    └── MAX(1, 2) = 2
        MIN = min(9, 2) = 2
```

```
MAX = max(5, 2) = 5
```

Alpha-Beta 从左到右搜索。左子树首先得到 5，因此根节点的

alpha = 5。随后在左侧 MIN 节点的第二个 MAX 子树中，先访问叶子

6，此时该 MAX 节点的值已经不小于 MIN 节点当前的

beta = 5，满足 alpha >= beta，因此叶子 9 被剪枝。该剪枝不影响

最终结果，访问节点数由 15 降为 14。

六、题目四：宝藏分配博弈

点的顺序为：

```
[A, B, T1, T2, T3, T4]
```

距离矩阵为：

```
[0, 12, 10, 4, 8, 8]
[12, 0, 8, 8, 4, 10]
[10, 8, 0, 6, 6, 12]
[4, 8, 6, 0, 6, 6]
[8, 4, 6, 6, 0, 6]
[8, 10, 12, 6, 6, 0]
```

宝藏价值分别为 T1=9、T2=7、T3=8、T4=7。玩家选择宝藏后的收益为 Gain = 宝藏价值 − 移动代价，局面评价值为 U = Score_A − Score_B。A 选择使 U 最大的策略，B 选择使 U 最小的策略。

Minimax 与 Alpha-Beta 在两个搜索深度下均得到相同的最优首步和收益差。

深度变化会改变决策，是因为短视搜索只考虑当前两回合，而完整搜索还会考虑后续位置对移动成本的影响。

深度	A 首步	关键应对过程	最终收益差
H=2	T2	A 获得 7−4=3，B 选择 T3 获得 8−4=4	−1
H=4	T3	最优对局为 A:T3、B:T2、A:T1、B:T4	3

在 H=2 时，T2 与 T3 的极小极大值实际同为 −1，程序按照宝藏的既定访问顺序首先保留 T2，因此输出 T2。到 H=4 时，搜索能够看到完整四回合：A 先取 T3 虽然即时收益为 0，但会获得更有利的后续位置，最终使收益差达到 3。这说明较深搜索能够避免只根据即时收益作出局部决策。

七、性能分析

1. A* 启发函数对比

启发函数	路径长度	扩展节点数	节点变化
h=0	8	16	基准
曼哈顿距离	8	14	减少 12.5%

两种方法都得到长度为 8 的最优路径，但曼哈顿距离将扩展节点数从 16 降至 14。每种方法重复运行 30 次，记录运行时间的平均值和标准差：

- h=0：平均 0.0214 ms，标准差 0.0042 ms；
- 曼哈顿启发式：平均 0.0205 ms，标准差 0.0031 ms。

由于测试地图较小，运行时间差异低于节点扩展差异，因此节点数仍作为主要效率指标。

2. Alpha-Beta 剪枝对比

题目三的小型测试树中，Minimax 访问 15 个节点，Alpha-Beta 访问 14 个节点，二者最优值均为 5。重复运行 30 次后，Minimax 平均耗时 0.0024 ms，标准差 0.0010 ms；Alpha-Beta 平均耗时 0.0033 ms，标准差 0.0016 ms。由于测试树规模较小，Alpha-Beta 的节点数减少 1 个，但计时差异没有统计意义。

综合题中的展开状态数如下：

搜索深度	Minimax	原始 Alpha-Beta	排序 Alpha-Beta
H=2	17	13	11
H=4	65	49	39

三种方法得到相同的最优首步和收益差。按照“当前收益较高的宝藏优先”进行节点排序后，H=2 的 Alpha-Beta 状态数从 13 降至 11，减少 15.4%；H=4 从 49 降至 39，减少 20.4%。相较 Minimax，排序后的 Alpha-Beta 在 H=4 时减少了 40% 的状态展开，说明良好的节点顺序能提高剪枝效率。

3. 节点访问顺序对比

在深度为 4、每个内部节点有 3 个子节点的测试树上，原始顺序访问 73 个节点，反转顺序访问 45 个节点，二者最优值均为 141。调整顺序后少访问 28 个节点，减少约 38.4%，说明节点访问顺序会显著影响 Alpha-Beta 的剪枝效率。

八、优化策略

1. 启发函数优化

普通迷宫使用曼哈顿距离作为启发函数，在保证最优性的同时减少节点扩展。

2. 距离矩阵对称性优化

本题的六个关键点 A、B、T1、T2、T3、T4 具有相同的进入代价，因此两点间最小移动代价对称。若分别计算所有有序点对，需要执行 $6\times5=30$ 次 A*；实现中只计算矩阵上三角的 $6\times5\div2=15$ 个无序点对，再镜像填充下三角，使 A* 调用次数减少 50%。这一优化只适用于距离确实对称的地图；如果不同关键点的进入代价不同，则不能直接镜像。

3. Minimax 记忆化搜索

宝藏 Minimax 使用 `lru_cache` 缓存已经计算过的状态，避免重复计算相同的玩家位置、剩余宝藏、累计得分和当前回合。

Alpha-Beta 没有直接缓存剪枝结果，因为剪枝返回值可能受到当前 `alpha` 和 `beta` 边界影响。

在题目给定的四宝藏场景中，H=2 和 H=4 的缓存命中次数均为 0，因为不同选择序列没有汇合到完全相同的状态，因此缓存没有减少该样例的展开量。为量化缓存效果，额外构造包含 6 个等价值宝藏的对称场景，并搜索完整 6 回合。无缓存时按排列树计算需要访问 1957 个节点；使用 `lru_cache` 后仅展开 487 个不同状态，产生 510 次缓存命中，状态展开减少 75.1%。

方法	节点或状态数	缓存命中	减少比例
无缓存 Minimax	1957	0	基准
记忆化 Minimax	487	510	75.1%

该实验说明，记忆化在具有大量对称或重复状态的博弈中效果明显；若不同路径几乎不会汇合到相同状态，缓存收益则会降低。

九、边界情况测试

测试场景	调用函数	返回值
普通迷宫无可行路径	`astar_path`	`[]`
加权地图无可行路径	`weighted_astar_cost`	`-1`
剩余宝藏全部不可达	两种宝藏博弈搜索	当前收益差，目标为 `None`

三类无解测试均通过。特别是在剩余宝藏全部不可达时，博弈搜索返回当前收益差和空目标，而不是错误的正负无穷值。

十、测试与可复现性

项目包含 14 个自动化测试，覆盖迷宫寻路、无解输入、加权地形、Minimax、Alpha-Beta、距离矩阵、综合题不同深度以及访问顺序对剪枝的影响。测试结果全部通过。

使用 coverage 统计得到 solution.py 覆盖率为 92%，项目整体覆盖率为 94%。项目提供 requirements.txt、verify.ps1 和 GitHub Actions 工作流，可通过以下命令复现实验：

```
python -m unittest test_solution.py -v
python run_experiment.py
python benchmark.py
```

十一、核心实验结果汇总

实验项目	方法或参数	结果	性能数据
迷宫寻路	A*，h=0	路径长度 8	扩展 16 个节点
迷宫寻路	A*，曼哈顿启发式	路径长度 8	扩展 14 个节点
加权地形	加权 A*	最小代价 7	避开高代价地形
博弈树	Minimax	最优值 5	访问 15 个节点
博弈树	Alpha-Beta	最优值 5	访问 14 个节点
宝藏博弈	H=2	T2，收益差 -1	17 / 13 / 11 个状态
宝藏博弈	H=4	T3，收益差 3	65 / 49 / 39 个状态

表中宝藏博弈的状态数按 Minimax / 原始 Alpha-Beta / 排序 Alpha-Beta 排列。实验结果表明，启发函数能够减少 A* 的节点扩展，Alpha-Beta 能在保持决策结果不变的同时减少博弈状态搜索。

十二、实验总结

本实验完成了普通与加权 A*、Minimax、Alpha-Beta 以及宝藏分配综合问题。实验说明：曼哈顿启发函数能够引导搜索；加权寻路必须累计实际地形代价；较深的博弈搜索能够避免短视决策；Alpha-Beta 的剪枝效率受到搜索深度和节点访问顺序影响。

在实现基础上，本实验还加入了 Minimax 状态缓存、距离矩阵对称性优化、访问顺序对照实验、自动化测试和持续集成。后续可进一步研究更大地图、更深博弈树以及动态节点排序策略。